Practical - 13

WEB DEVELOPMENT FRAMEWORKS (ITUE203)

SEMESTER: 3RD, 2026-27

ID: 25DCE110

Practical Title: PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Theory

Answers to key questions

1. Do PHP endpoints return valid JSON responses?

Yes. PHP endpoints return JSON data using json_encode() and set the Content-Type header to application/json.

2. Are GET and POST requests handled properly?

Yes. GET retrieves records, while POST submits data to create records. Update and delete operations can use POST with an action field or other explicitly supported HTTP methods.

3. Are loading, success, and error states shown on the frontend?

Yes. JavaScript displays loading messages while requests are running and success or error messages after the response.

4. Is input validation performed before database operations?

Yes. PHP validates the received data on the server before executing database queries.

Implementation/ Output/Result

Tags:

Demo code:

```
<?php
header("Content-Type: application/json");

$tasks = [
    ["id" => 1, "title" => "Learn PHP"],
    ["id" => 2, "title" => "Learn JavaScript"]
];

if ($_SERVER["REQUEST_METHOD"] === "GET") {
    echo json_encode($tasks);
} elseif ($_SERVER["REQUEST_METHOD"] === "POST") {
    $data = json_decode(file_get_contents("php://input"), true);
    $title = trim($data["title"] ?? "");

    if ($title === "") {
        http_response_code(400);
        echo json_encode(["message" => "Title is required"]);
    } else {
        echo json_encode([
            "message" => "Demo task received",
            "title" => $title
```

```
        ]);
    }
} else {
    http_response_code(405);
    echo json_encode(["message" => "Method not supported in this demo"]);
}
?>



<!DOCTYPE html>
<html>
<head>
    <title>Fetch API Demo</title>
</head>
<body>
    <h2>Task Manager</h2>

    <input id="task" placeholder="Enter task">
    <button onclick="addTask()">Add Task</button>

    <p id="message"></p>
    <ul id="list"></ul>

    <script>
    async function loadTasks() {
        const response = await fetch("api.php");
```

```
    const tasks = await response.json();
    const list = document.getElementById("list");
    list.replaceChildren();

    tasks.forEach(task => {
        const li = document.createElement("li");
        li.textContent = task.title;
        list.appendChild(li);
    });
}

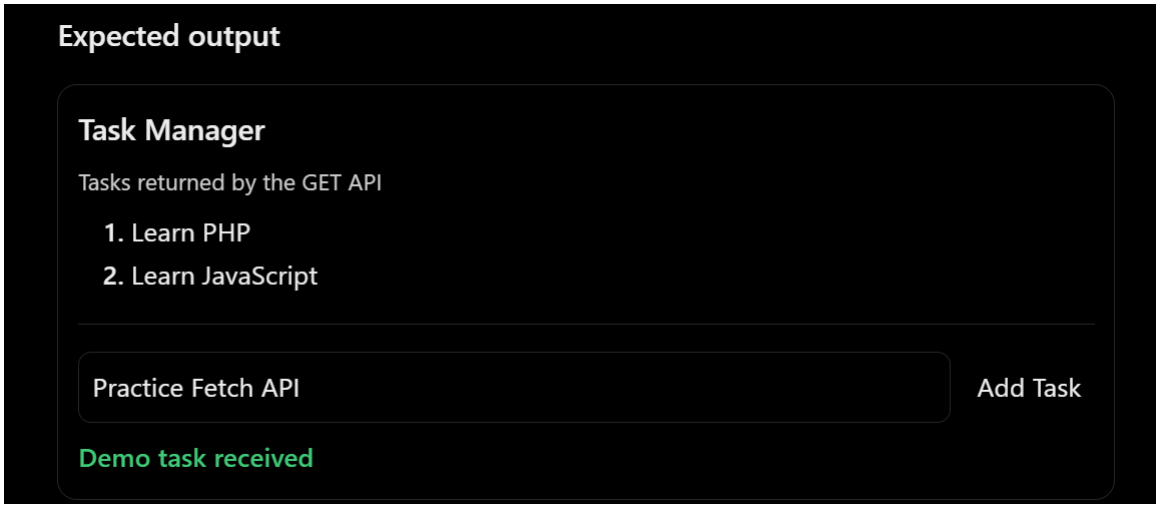
async function addTask() {
    const title = document.getElementById("task").value.trim();
    const message = document.getElementById("message");

    if (!title) {
        message.textContent = "Please enter a task";
        return;
    }

    const response = await fetch("api.php", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({title: title})
    });
```

```
        const data = await response.json();
        message.textContent = data.message;
    }

    loadTasks();
    </script>
</body>
</html>
```



This example covers GET and POST only. Full Practical 13 also requires update and delete endpoints.

- Created PHP endpoints to communicate with the database.
- Returned data in JSON format.
- Used GET to retrieve student records.
- Used POST to add student records.
- Used JavaScript Fetch API to send asynchronous requests.
- Displayed loading, success, and error messages.

- Validated input before database operations.
- Used prepared statements to improve query safety.

GitHub Repository Link:

https://github.com/kanan2007s-sys/Web-development-/tree/main

Learning Outcomes:

Learned to integrate JavaScript Fetch API with PHP JSON endpoints for asynchronous database operations without full-page reloads.

Faculty Remarks: